

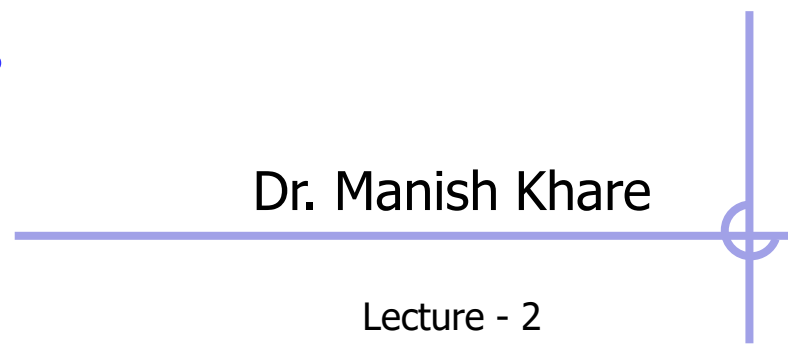
Hash Tables
Trees
Sets
Stack
Linked Lists
Graphs
Sets
Queues
DATA STRUCTURES
Linked Lists
Queues
Hash Tables
Sets
Array
Stack
Queues
Trees
Linked Lists
Graphs
Sets
Array

Data Structures

IT623

Dr. Manish Khare

Lecture - 2



Algorithm



Definition of Algorithm?

Definition of Algorithm?

- Algorithm is a step-by-step procedure, which defines a set of instructions to be executed in a certain order to get the desired output. Algorithms are generally created independent of underlying languages, i.e. an algorithm can be implemented in more than one programming language.
- From the data structure point of view, following are some important categories of algorithms –
 - Search – Algorithm to search an item in a data structure.
 - Sort – Algorithm to sort items in a certain order.
 - Insert – Algorithm to insert item in a data structure.
 - Update – Algorithm to update an existing item in a data structure.
 - Delete – Algorithm to delete an existing item from a data structure.

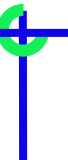
Algorithm

- 
- Definition of Algorithm?
 - Characteristics of an Algorithm?

Characteristics of an Algorithm?

- Not all procedures can be called an algorithm. An algorithm should have the following characteristics –
- **Unambiguous** – Algorithm should be clear and unambiguous. Each of its steps (or phases), and their inputs/outputs should be clear and must lead to only one meaning.
- **Input** – An algorithm should have 0 or more well-defined inputs.
- **Output** – An algorithm should have 1 or more well-defined outputs, and should match the desired output.
- **Finiteness** – Algorithms must terminate after a finite number of steps.
- **Feasibility** – Should be feasible with the available resources.
- **Independent** – An algorithm should have step-by-step directions, which should be independent of any programming code.

Algorithm

- 
- Definition of Algorithm?
 - Characteristics of an Algorithm?
 - How to Write an Algorithm?

How to Write an Algorithm?

- There are no well-defined standards for writing algorithms. Rather, it is problem and resource dependent. Algorithms are never written to support a particular programming code.
- As we know that all programming languages share basic code constructs like loops (do, for, while), flow-control (if-else), etc. These common constructs can be used to write an algorithm.
- We write algorithms in a step-by-step manner, but it is not always the case. Algorithm writing is a process and is executed after the problem domain is well-defined. That is, we should know the problem domain, for which we are designing a solution

Algorithm Example

➤ Design an algorithm to add two numbers and display the result.

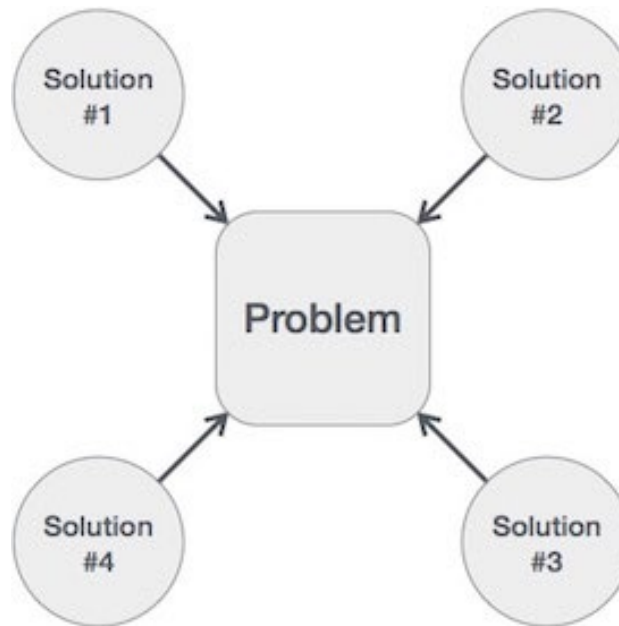
- step 1 – START
- step 2 – declare three integers a, b & c
- step 3 – define values of a & b
- step 4 – add values of a & b
- step 5 – store output of step 4 to c
- step 6 – print c
- step 7 – STOP

Algorithm Example

- step 1 – START ADD
- step 2 – get values of a & b
- step 3 – $c \leftarrow a + b$
- step 4 – display c
- step 5 – STOP

Algorithm

- We design an algorithm to get a solution of a given problem.
A problem can be solved in more than one ways.



➤ **Algorithm:**

- Outline, the essence of a computational procedure, step-by-step instructions

➤ **Program:**

- An implementation of an algorithm in some programming language

➤ **Data Structure:**

- **Organization** of data needed to solve the problem

Algorithm Analysis

➤ Efficiency of an algorithm can be analyzed at two different stages, before implementation and after implementation. They are the following

- A Priori Analysis
- A Posterior Analysis

➤ Algorithm analysis deals with the execution or running time of various operations involved. The running time of an operation can be defined as the number of computer instructions executed per operation

Algorithm Analysis

- **A Priori Analysis** – This is a theoretical analysis of an algorithm. Efficiency of an algorithm is measured by assuming that all other factors, for example, processor speed, are constant and have no effect on the implementation.
- **A Posterior Analysis** – This is an empirical analysis of an algorithm. The selected algorithm is implemented using programming language. This is then executed on target computer machine. In this analysis, actual statistics like running time and space required, are collected.

Algorithm Complexity

- Suppose **X** is an algorithm and **n** is the size of input data, the time and space used by the algorithm X are the two main factors, which decide the efficiency of X.
- **Time Factor** – Time is measured by counting the number of key operations such as comparisons in the sorting algorithm.
- **Space Factor** – Space is measured by counting the maximum memory space required by the algorithm.
- The complexity of an algorithm **f(n)** gives the running time and/or the storage space required by the algorithm in terms of **n** as the size of input data.

Space Complexity

➤ Space complexity of an algorithm represents the amount of memory space required by the algorithm in its life cycle. The space required by an algorithm is equal to the sum of the following two components –

- A fixed part that is a space required to store certain data and variables, that are independent of the size of the problem. For example, simple variables and constants used, program size, etc.
- A variable part is a space required by variables, whose size depends on the size of the problem. For example, dynamic memory allocation, recursion stack space, etc.

- 
- Space complexity $S(P)$ of any algorithm P is $S(P) = C + S(I)$, where C is the fixed part and $S(I)$ is the variable part of the algorithm, which depends on instance characteristic I . Following is a simple example that tries to explain the concept –
 - Algorithm: SUM(A, B)
 - Step 1 - START
 - Step 2 - $C \leftarrow A + B + 10$
 - Step 3 - Stop
 - Here we have three variables A , B , and C and one constant. Hence $S(P) = 1 + 3$. Now, space depends on data types of given variables and constant types and it will be multiplied accordingly.

Time Complexity

- Time complexity of an algorithm represents the amount of time required by the algorithm to run to completion. Time requirements can be defined as a numerical function $T(n)$, where $T(n)$ can be measured as the number of steps, provided each step consumes constant time.
- For example, addition of two n -bit integers takes n steps. Consequently, the total computational time is $T(n) = c * n$, where c is the time taken for the addition of two bits. Here, we observe that $T(n)$ grows linearly as the input size increases

Asymptotic Analysis

- Asymptotic analysis of an algorithm refers to defining the mathematical boundation/framing of its run-time performance.
- Using asymptotic analysis, we can very well conclude the best case, average case, and worst case scenario of an algorithm.
- Asymptotic analysis is input bound i.e., if there's no input to the algorithm, it is concluded to work in a constant time. Other than the "input" all other factors are considered constant.

Asymptotic Analysis

- 
- Usually, the time required by an algorithm falls under three types –
- **Best Case** – Minimum time required for program execution.
 - **Average Case** – Average time required for program execution.
 - **Worst Case** – Maximum time required for program execution.

Asymptotic Analysis

- Asymptotic analysis refers to computing the running time of any operation in mathematical units of computation.
- For example, the running time of one operation is computed as $f(n)$ and may be for another operation it is computed as $g(n^2)$.
- This means the first operation running time will increase linearly with the increase in n and the running time of the second operation will increase exponentially when n increases.
- Similarly, the running time of both operations will be nearly the same if n is significantly small.